

Lists

HOW WOULD YOU STORE THIS?

- Your class has 30 students, and each one has a test score
- You want the class average
- With what we know so far, that means 30 separate variables
- A list gives the whole collection one name

```
score1 = 86
score2 = 94
score3 = 75
# ... 27 more of these
total = score1 + score2 + ...
```



one name for the
whole class

```
scores = [84, 94, 75]
print(sum(scores) / len(scores))
```

85.0

A LIST IS A SEQUENCE

- A list is a sequence of values, called elements
- Make one with square brackets, with commas between the elements
- Elements can be any type, even other lists
- `[]` is the empty list, and `len` gives the number of elements

```
numbers = [42, 123]
cheeses = ["Brie", "Edam", "Feta"]
mixed = ["spam", 2.0, 5, [10, 20]]
empty = []
print(len(cheeses))
print(len(mixed))
print(len(empty))
```

3
4
0

INDEXING AND SLICING

- Everything you know about string indexes works on lists
- Indexes start at 0, and negative indexes count from the end
- A slice of a list is a new list
- `in` checks whether a value is an element of the list

```
letters = ["a", "b", "c", "d"]  
print(letters[0])  
print(letters[-1])  
print(letters[1:3])  
print("c" in letters)
```

```
a  
d  
['b', 'c']  
True
```

LIST OPERATORS

- + joins two lists into a new list
- * repeats a list
- sum, min, and max work on a list of numbers

```
t1 = [1, 2]
t2 = [3, 4]
print(t1 + t2)
print(["spam"] * 3)
print(sum(t1), min(t1), max(t2))
```

```
[1, 2, 3, 4]
['spam', 'spam', 'spam']
3 1 4
```

LIST METHODS

- `append` adds one element to the end
- `extend` adds every element of another list
- `pop` removes the element at an index and returns it
- `remove` removes a value; it's a `ValueError` if the value isn't there

```
t = ["a", "b", "c"]
t.append("d")
t.extend(["e", "f"])
print(t)
print(t.pop(1))
t.remove("a")
print(t)
```

```
['a', 'b', 'c', 'd', 'e', 'f']
b
['c', 'd', 'e', 'f']
```

LOOPING THROUGH A LIST

- A `for` loop visits each element in order
- It's the same pattern as looping over the characters of a string
- A loop over an empty list never runs its body

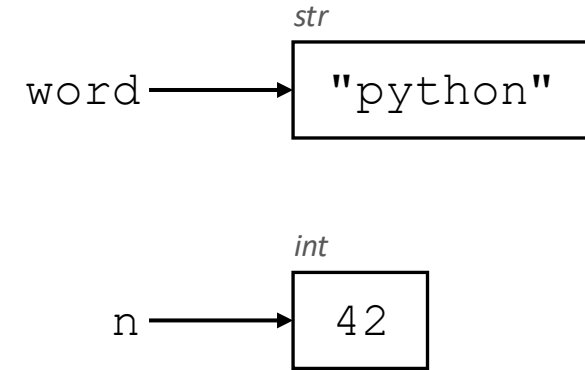
```
cheeses = ["Brie", "Edam", "Feta"]  
for cheese in cheeses:  
    print(cheese)
```

```
Brie  
Edam  
Feta
```

NAMES AND OBJECTS

- Every value in Python is an object in memory
- A variable is a name that refers to an object
- Assignment points a name at an object; it never copies anything
- We draw that reference as an arrow

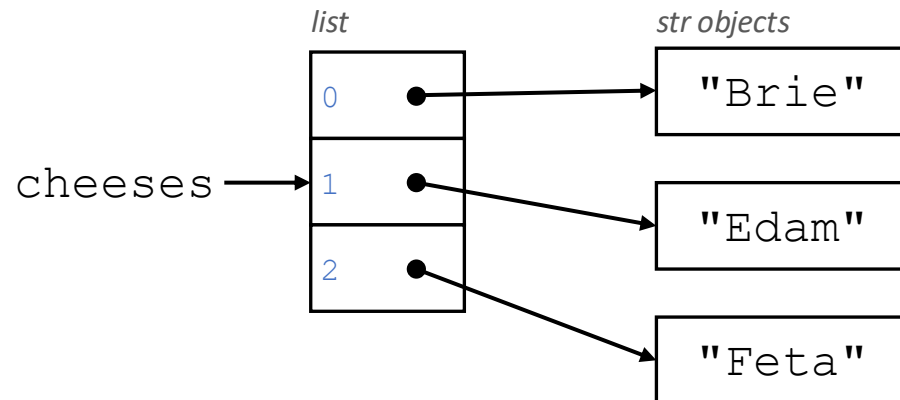
```
word = "python"  
n = 42
```



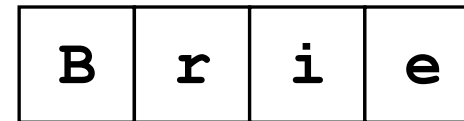
WHAT A LIST LOOKS LIKE INSIDE

- A list doesn't hold its elements directly
- It holds references (arrows) to them
- A string is different: its characters are packed together inside one object

```
cheeses = ["Brie", "Edam", "Feta"]
```

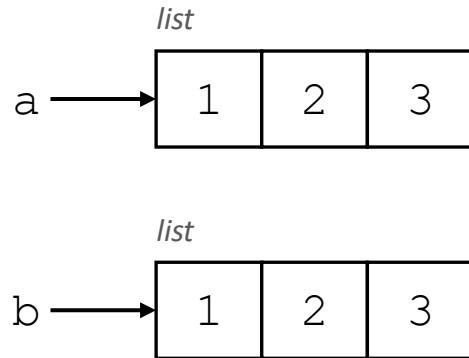


inside the "Brie" str object



SAME VALUE, OR SAME OBJECT?

- `==` asks: do these have the same value? (equivalent)
- `is` asks: are these the same object? (identical)
- `id(x)` gives an object's identity number
- Two lists built separately are two different objects



```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)
print(a is b)
print(id(a), id(b))
```

True

False

4371204416 4371377920

(your id numbers will differ)

REMEMBER THIS?

- Last time: strings are immutable, so you can't change a character
- The same move on a list works: lists are mutable

string

```
word = "python"  
word[0] = "J"
```

`TypeError: 'str' object does
not support item assignment`

list

```
letters = ["p", "y", "t", "h", "o", "n"]  
letters[0] = "J"  
print(letters)
```

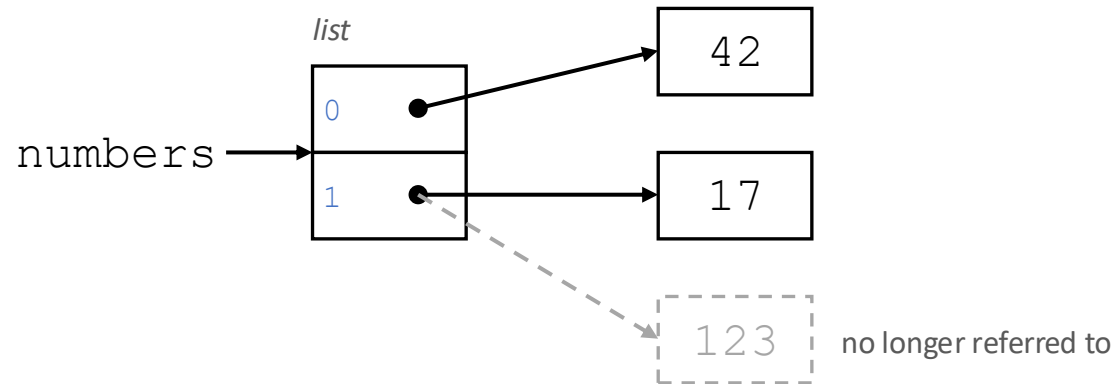
```
['J', 'y', 't', 'h', 'o', 'n']
```

LISTS ARE MUTABLE

- Assigning to an index replaces that element
- It's still the same list object; one of its arrows moved
- 123 wasn't changed either; the list just stopped referring to it

```
numbers = [42, 123]  
numbers[1] = 17  
print(numbers)
```

[42, 17]



WHY word = word.upper()?

- What happens when we run:

```
word = word.upper()
```

- `upper` can't change the string, because strings are immutable
- So it builds a new string and returns it
- If you don't save the result, it's gone

```
word = "python"  
word.upper()  
print(word)  
word = word.upper()  
print(word)
```

```
python  
PYTHON
```

LIST METHODS CHANGE THE LIST

- List methods change the list in place
- Most of them return `None`: there's no new list to hand back
- That's the opposite of string methods

```
t = [3, 1, 2]
result = t.append(4)
print(t)
print(result)
```



nothing to hand back

```
[3, 1, 2, 4]
None
```

`+=` ON A LIST VS. A STRING

- On a list, `+=` changes the list in place: same object, and every alias sees it
- On a string, `+=` builds a new string and moves the name to it

list

```
a = [1, 2]
b = a
print(id(a))
a += [3]
print(id(a))
print(b)
```

```
4371204416
4371204416
[1, 2, 3]
```

string

```
s = "hi"
t = s
print(id(s))
s += "!"
print(id(s))
print(t)
```

```
4370519984
4371377776
hi
```

(your id numbers will differ)

A BUG YOU'LL WRITE THIS WEEK

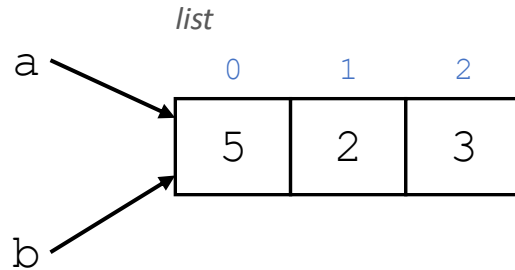
- Strings taught you the pattern `x = x.method()`
- With a list method, that saves `None` into `t`
- The list is lost, and the next line fails
- If an error mentions `NoneType`, look for a list method whose result you saved

```
t = [1, 2, 3]
t = t.remove(3)
t.remove(2)
```

```
AttributeError: 'NoneType'
object has no attribute
'remove'
```

ALIASING

- `b = a` doesn't copy the list; it gives the same object a second name
- An object with more than one name is aliased
- Change it through one name and the other name sees the change

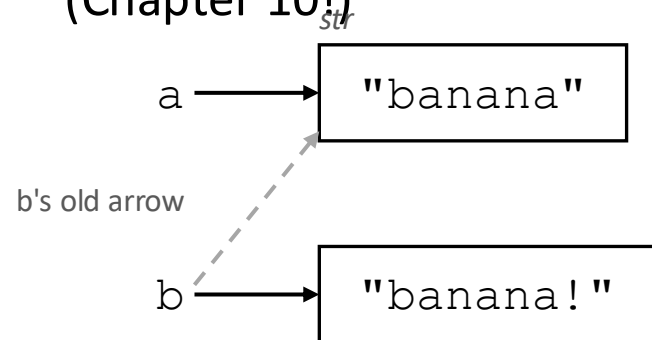


```
a = [1, 2, 3]
b = a
b[0] = 5
print(a)
print(a is b)
```

```
[5, 2, 3]
True
```

ALIASING AND STRINGS

- Aliasing a string is harmless, because nobody can change it
- "Changing" `b` makes a new string; `a` still refers to the old one
- That's one big reason strings are immutable: they're safe to share
- Immutable values can also be dictionary keys (Chapter 10!)



```
a = "banana"
b = a
b = b + "!"
print(a)
print(b)
```

```
banana
banana!
```

MAKING A COPY

- Slice the whole list with `a[:]`, or call `list(a)`
- Either one gives a new list with the same elements
- Now a change through one name doesn't affect the other

```
a = [1, 2, 3]
b = a[:]
c = list(a)
b[0] = 5
print(a)
print(b)
print(a is c)
```

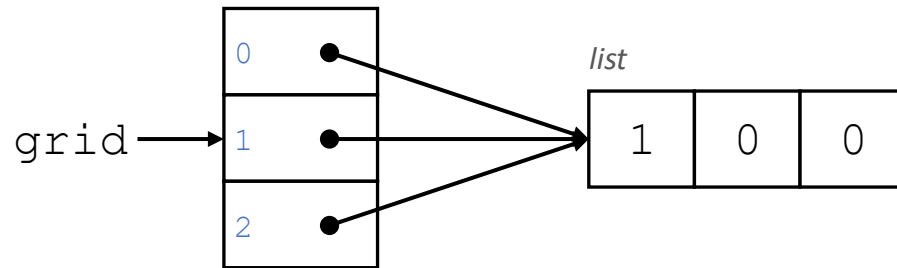
```
[1, 2, 3]
[5, 2, 3]
False
```

COPIES ARE SHALLOW

- A copy is a new list of the same references
- * on a list copies references too
- Here all three rows are the same inner list

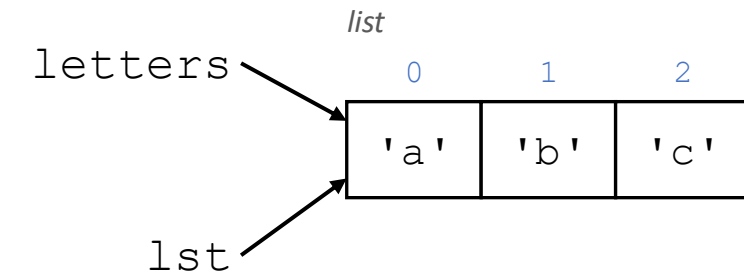
```
grid = [[0] * 3] * 3  
grid[0][0] = 1  
print(grid)
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```



PASSING A LIST TO A FUNCTION

- The function gets a reference to the caller's list, not a copy
- The parameter is one more alias
- If the function changes the list, the caller sees the change



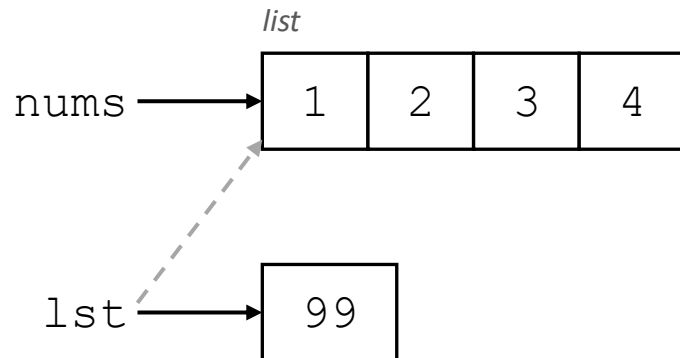
(inside `pop_first`)

```
def pop_first(lst):  
    return lst.pop(0)  
  
letters = ["a", "b", "c"]  
print(pop_first(letters))  
print(letters)
```

```
a  
['b', 'c']
```

CHANGING VS. REASSIGNING

- Changing the list inside a function: the caller sees it
- Assigning a new list to the parameter: the caller doesn't
- Assignment only moves the local name's arrow



```
def f(lst):  
    lst.append(4)  
    lst = [99]
```

```
nums = [1, 2, 3]  
f(nums)  
print(nums)
```

`[1, 2, 3, 4]`

LISTS AND STRINGS

- `list(s)` breaks a string into characters
- `s.split()` breaks it into words;
`s.split("-")` splits at a delimiter
- `join` goes the other way: call it on the delimiter, and pass the list

```
print(list("spam"))
words = "pinning for the fjords".split()
print(words)
print(" ".join(words))
letters = ["J", "y", "t", "h", "o", "n"]
print("".join(letters))
```

```
['s', 'p', 'a', 'm']
['pinning', 'for', 'the', 'fjords']
pinning for the fjords
Jython
```

SORTING

- `sorted` returns a new, sorted list
- The original is left unchanged
- It works on any sequence, including strings
- Use `join` to turn the result back into a string

```
scramble = ["c", "a", "b"]
print(sorted(scramble))
print(scramble)
print(sorted("letters"))
print("".join(sorted("letters")))
```

```
['a', 'b', 'c']
['c', 'a', 'b']
['e', 'e', 'l', 'r', 's', 't', 't']
eelrstt
```

PREDICT THE OUTPUT

- Work these out on paper before running them
- Then check your answers in Colab
- Draw the arrows; it helps!
- Watch the third one closely

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)

s = "cat"
t = s
t += "s"
print(s)

def add_one(lst):
    lst.append(1)
    lst = [0]
nums = [5]
add_one(nums)
print(nums)
```